Validador `express-validator`

- Validador `express-validator`
  - Instalación y uso básico
  - Validadores
  - Saneadores
  - Encadenamiento
  - Array de funciones `check()`
  - Validación y Saneamiento
  - Mensaje de error personalizado
  - Encadenamiento
  - Cadena de validación constante
  - Función de validación:
  - Módulo validador
  - Función manejadora de errores
  - Validador Personalizado
  - Esquema de validación

Instalación y uso básico

```
$ npm install express-validator
```

Requerimos el objeto **`check`** de **`express-validator`** Y hacemos llamadas a **`check()`** pasándole el nombre del parámetro a validar. La función **`check()`** tiene métodos. Estos métodos están definidos en la librería **`validator.js`**

```
const { check, validationResult } = require('express-validator')

app.post('/usuario', check('edad').isNumeric(), (req, res) => {
      //Procesar solicitud
})
```

Verificar el resultado del proceso de validación con la función **`validationResult()`**

```
app.post('/usuario', check('edad').isNumeric(), (req, res) => {
      const result = validationResult(req);

if (result.isEmpty()) {
          /* Validación OK */
      } else {
          /* Error de Validación */
      }
})
```

Validador `express-validator`

Validadores

`https://github.com/validatorjs/validator.js`

Método	Valida que la entrada	Método	Valida que la entrada ....
`contains()`	..contiene el valor especificado	`isIP()`	..sea una IP versión 4 o 6
`equals()`	..sea igual al valor especificado	`isIn()`	..esté dentro del array de valores especificado
`isAlpha()`	..contenga solo letras	`isInt()`	..sea un entero
`isAlphanumeric()`	..sea alfanumérica	`isJSON()`	..sea un Json válido
`isAscii()`	..contenga solo caracteres Ascii	`isLength()`	..sea de determinada longitud
`isBase64()`	..este codificada como base64	`isLowercase()`	..esté en minúsculas
`isBoolean()`	..sea un valor booleano	`isNumeric()`	..contenga solo números
`isDecimal()`	..represente un número decimal	`isUppercase()`	..este en mayúsculas
`isEmpty()`	..que la entrada tenga una longitud cero	`isWhitelisted()`	..contra uns lista de caracteres permitida
`isFQDN()`	..sea un nombre de dominio calificado		
`isFloat()`	..sea un flotante		
`isHash()`	..sea un algoritmo Hash		
`isHexColor()`	..sea un valor de color hexadecimal		

Validador `express-validator`

Saneadores

Método	Que realizara en la entrada?
`blacklist()`	Borrará caracteres que aparezcan en la *lista negra*
`whitelist()`	Borrará caracteres que no aparezcan en la lista de permitidos
`escape()`	Convierte los caracteres `< > & ' " /` en entidades HTML
`unescape()`	Sustituirá las entidades HTML en su carácter correspondiente
`ltrim()`	Borrará caracteres en blanco al inicio
`rtrim()`	Borrará caracteres en blanco al final
`stripLow()`	Borrará caracteres ASCII de control (invisibles)
`toBoolean()`	Convertirá a Booleano.
`toDate()`	Convertirá a `date` o `null` si la entrada no es fecha
`toFloat()`	Convertirá a `float`, o `NaN` si la entrada no es flotante
`toInt()`	Convertirá a `integer`, o `NaN` si la entrada no es entera

Validador `express-validator`

Encadenamiento

Todas las llamadas a la función **`check()`** se pueden encadenar:

```
app.post('/usuario', check('nombre').isAlpha().islength({min:3}), (req, res) => {
      const result = validationResult(req);

if (result.isEmpty()) {
          /* Validación OK */
      } else {
          /* Error de Validación */
    }
})
```

Validador `express-validator`

Array de funciones `check()`

A la función manejadora de la petici≤n le podemos pasar un array de funciones **`check()`****

```
const express = require('express');
const { check, validationResult } = require('express-validator');
const app=express();
app.use(express.json());

app.post("/usuario",[ check('edad').isNumeric(),
                      check('correo').isEmail(),
        ],(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});

app.listen(8080,(req,res)=>{
    console.log("Servidor express escuchando: 8080");
});
```

Validador `express-validator`

Validación y Saneamiento

Realizando un proceso de saneamiento después de cada validación

```
app.post('/usuario', [check('nombre').isLength({ min: 3 }).trim().escape(),
                      check('correo').isEmail().normalizeEmail(),
                      check('edad').isNumeric().trim().escape()
        ],(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Mensaje de error personalizado

```
const express = require('express');
const { check, validationResult } = require('express-validator');
const app=express();

app.use(express.json());
app.post("/usuario",[ check('login','Login debe ser de 5 a 10 caracteres').isLength({min:5, max:10}),
                      check('edad','Edad Debe ser Numérica').isNumeric(),
                      check('correo',"El correo es invalido").isEmail(),
        ],(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Encadenamiento

Encadenar cada validación con su correspondiente mensaje de error

```
const express = require('express');
const { check, validationResult } = require('express-validator');
const app=express();

app.use(express.json());

app.post("/usuario",[ check('login').isLength({min:5, max:10}).withMessage("Longitud incorrecta")
                                    .isAlphanumeric().withMessage('Login debe contener numero y letras'),
                      check('edad').isNumeric().withMessage("Edad debe ser un numero")
                      check('correo',"El correo es invalido").isEmail()
        ],(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Cadena de validación constante

Creando una constante con la **`cadena de validación`**😗*

```
const express = require('express');
const { check, validationResult } = require('express-validator');
const app=express();

app.use(express.json());

const validaUsuario = [ check('login','Login debe ser de 5 a 10 caracteres').isLength({min:5, max:10}),
                        check('edad','Edad Debe ser Numérica').isNumeric(),
                        check('correo',"El correo es invalido").isEmail()];

app.post("/usuario",validaUsuario,(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Función de validación:

Podemos crear una función que regrese la cadena de validación
para que el código quede mas limpio y fácil de leer:

```
const express = require('express');
const { check, validationResult } = require('express-validator');
const app=express();

app.use(express.json());
const validaUsuario = () => {
    return [ check('login','Login debe ser de 5 a 10 caracteres').isLength({min:5, max:10}),
             check('edad','Edad Debe ser Numérica').isNumeric(),
             check('correo',"El correo es invalido").isEmail()]
}

app.post("/usuario",validaUsuario(),(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Módulo validador

Creando un módulo validador e importándolo en el servidor express.

```
const { check } = require('express-validator');                                 //MODULO VALIDADOR

function validaUsuario() {
    return [ check('login','Login debe ser de 5 a 10 caracteres').isLength({min:5, max:10}),
             check('edad','Edad Debe ser Numérica').isNumeric(),
             check('correo',"El correo es invalido").isEmail()]
}
module.exports.validaUsuario=validaUsuario;            //Exportamos validación
```

```
const express = require('express');                                             //SERVIDOR PRINCIPAL
const { validationResult } = require('express-validator');
const  validadores = require('./validadores.js');     //Importamos validación
const app=express();

app.use(express.json());
app.post("/usuario",validadores.validaUsuario(),(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Función manejadora de errores

Redireccionando a función manejadores de errores:

```
const express = require('express');
const { validationResult } = require('express-validator');
const  validadores = require('./validadores.js');
const app=express();

app.use(express.json());
app.post("/usuario",validadores.validaUsuario(),(req,res)=>{
    const result = validationResult(req);

    if (result.isEmpty()) {
        res.json({mensaje:"Respuesta a petición post"});
    } else {
        let err = new Error("Error Validación");//construir un mensaje de error a partir de result
        next(err);
    }
});

//Función Manejadora de Errores
app.use((err,req,res,next)=>{
    res.status(400).json({error:err.message});
});

app.listen(8080,(req,res)=>{ console.log("Servidor Express escuchando 8080"); });
```

Validador express-validator

Validador Personalizado

Si no nos es suficiente con los métodos que ya vienen definidos
podemos definir unos propio con la función **`custom()`**

```
const express = require('express');
const { check, validationResult } = require('express-validator');
const app=express();

app.use(express.json());
app.post("/usuario",[ check('login','Login debe ser de 5 a 10 caracteres').isLength({min:5, max:10}),
                      check('edad','Edad Debe ser Numérica').isNumeric(),
                      check('correo',"El correo es invalido").isEmail(),
                      check('edad').custom((value)=>{
                                          if (value<18) { throw new Error("Menor de edad")}
                                          else { return true }
                                        })
        ],(req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`

Esquema de validación

En lugar de usar la cadena de validación podemos usar la función **`checkSchema()`** y pasarle un objeto:**

```
const express = require('express');
const { checkSchema, validationResult } = require('express-validator');
const app=express();

app.use(express.json());

app.post("/usuario",checkSchema({
            'login':  {isLength: {min:5, max:10}},
            'edad':   {isNumeric:{}},
            'correo': {isEmail:  {}}
        }),
        (req,res)=>{
        const result = validationResult(req);

        if (result.isEmpty()) {
            res.json({mensaje:"Respuesta a petición post"});
        } else {
            res.json(result);
        }
});
```

Validador `express-validator`